

KMIP on PKCS#11

Delivery Plan — a complete KMS with a REST control plane

26 steps in 9 stages · design, implementation and test coverage for each

Baseline	e44a2ef — 816 tests green, 41 of 53 KMIP 2.1 operations
Starting position	50 of 82 assessed features are short of full coverage
This plan closes	50 of those 50, across 26 steps
Deliberately not closed	0 — supplier, procurement, external event, or out of product scope
Method	Every step: a design, an implementation, a test plan, and a gate that must be demonstrated
Date	12 September 2026

1 How this plan works

Phases 0 to 5 of this project succeeded because each had a gate — a condition someone could watch you meet — rather than a checklist. This plan keeps that structure and adds the two things the request asks for: a design deliverable before implementation, and a named test plan rather than a promise of coverage.

Each step carries five things

Part	What it means
Objective	One sentence. If it needs two, the step is two steps.
Design	Written and reviewed before code. Names the interfaces, the data, and the decisions that are hard to reverse.
Implementation	What changes, and equally what deliberately does not.
Tests	Named cases, not a coverage percentage. Every step adds regressions for the behaviour it introduces, and the existing suite must stay green.
Gate	A demonstrable condition. The step is finished when the gate is shown, not when the code is written.

Rules every step inherits

- Tests run live against a real SoftHSM2 token. The token is never mocked — the defects worth catching are in the boundary.
- The suite stays green at every step. A step that leaves it red is not finished, and the next step does not start.
- Anything that could not be tested in this environment is recorded as untested rather than shipped as though it had been.
- Documents are regenerated, not hand-edited, and the gap matrix's ASSESSED_AT is bumped whenever coverage changes.

The arithmetic in this document is checked at build time against generate_kms_gap_matrix.py. If a step claims a feature the matrix does not list as open, claims one twice, or an open feature appears in neither the plan nor the deferred list, the generator refuses to build. That is deliberate: a plan whose numbers do not reconcile with the assessment it derives from is worse than no plan.

2 What the plan closes

Of 82 assessed features, 50 are short of full coverage today — 18 partial and 32 absent. The 26 steps close 50 of them. The remaining 0 are listed in section 5 with the reason each is not engineering work this plan can schedule.

Step	Stage	Title	Features closed
1	A	Service layer and guard	—
2	A	Query and reporting layer	2
3	B	HTTP transport, sessions and service accounts	3
4	B	Read endpoints	—
5	B	Write endpoints, and the governance gaps	4
6	B	Administration, OpenAPI, separation of duties	3
7	B	Web console and self-service	2
8	C	Enterprise identity and policy	2
9	C	Audit externalisation, compliance and ceremony	7
10	D	Storage abstraction and PostgreSQL	1
11	D	High availability, failover and deployment	6
12	D	Multi-tenancy	2
13	E	KMIP JSON and XML encodings	1
14	E	Certificate discovery and expiry monitoring	1
15	F	Cloud BYOK	1
16	F	External key store — XKS and EKM	1
17	F	PKCS#11 provider for applications	1
18	F	Tokenization and format-preserving encryption	1
19	G	Vendor interoperability validation	2
20	G	OASIS interoperability certification	1
21	G	Certificate authority integration	1
22	G	Secrets manager integration	1
23	H	KMIP 3.0	1
24	H	Post-quantum and full algorithm breadth	2
25	I	Validated hardware migration	3
26	I	Confidential computing deployment	1

The 9 stages

Stage	Name	Character
A	Foundation	No new attack surface. Makes the rest safe to build.
B	The REST control plane	The API, then the console that uses it.
C	Enterprise fit	What an auditor and a security team ask for.
D	Scale, availability and tenancy	Needs infrastructure to verify.
E	Protocol and estate reach	Additive, independent of D, verifiable here.
F	Cloud and application reach	New surfaces for callers this server cannot serve today.
G	Interoperability and adjacent products	Meeting other implementations, and integrating rather than rebuilding.
H	Protocol 3.0 and post-quantum	In-protocol work, then algorithms only a capable token can run.
I	Assurance on real hardware	Gated on procurement. Fully specified, cannot start here.

Steps 1 and 2 close nothing and are still the two most important. They are the reason the ten that follow can be built without each one re-litigating authorization, and the reason a REST endpoint cannot quietly become a path around dual control.

3 Sequence and dependencies

The order is dictated by dependency, not by visibility. Three constraints fix most of it.

- Step 1 precedes every other step. Authorization, dual control and audit currently live inside the KMIP dispatcher; a REST layer built before they move would bypass all three, and the bypass would be silent.
- Step 10 precedes step 11, which precedes step 12. SQLite has no multi-writer story, so clustering waits on PostgreSQL, and tenancy is only worth building on something that can survive a node failure.
- Step 3 precedes 4, 5 and 6, which precede 7. The console is a client of the API; building it earlier means building it twice.
- Steps 8 and 9 depend only on stage B and can run in parallel with each other, or with stage D if there are two teams.

Where the plan needs infrastructure it does not have

Step	Needs	If unavailable
10	A PostgreSQL instance in CI	The abstraction can be built and the SQLite path kept green, but the PostgreSQL implementation cannot be claimed as verified.
11	Multiple nodes and a second HSM token	Failover must not be marked done. An untested failover path invites exactly the reliance it cannot support.
11	Network access to a container registry	The image and CI workflow have still never been executed. This step is the first opportunity to fix that.
7	A browser in CI for end-to-end tests	Playwright and Chromium are available in this environment, so this one is not blocked.

4 The steps

Sizes are estimates for planning, not commitments. The test counts are the more meaningful figure: they say what the step has to prove.

Step 1 — Service layer and guard

Stage A · ~600 lines moved, ~15 new tests. No new dependencies.

Make authorization, dual control, audit and metrics impossible to bypass — before a second transport exists to bypass them.

Impact on the existing design

- OperationDispatcher loses its cross-cutting logic and becomes a codec: decode, call the service, encode.
- access_control.py and dual_control.py are invoked from one place instead of being wired into a transport.
- No schema change, no configuration change, nothing on the wire changes.

Design

- A new kmip_pkcs11/service/ package: CallContext (identity, peer address, transport), a guard() context manager, and KeyService / AdminService facades over the existing handlers and store.
- The guard runs the role allowlist, then dual control, then the operation, then audit and metrics — the exact order dispatch() uses today.
- OperationDispatcher shrinks to: decode TTLV, call the service, encode TTLV.

Implementation

- Move check_operation_allowed, dual_control_enforce, _audit and _record_metric out of the dispatcher into the guard. Operation handlers are not touched.
- Every service method takes a CallContext as its first argument, so a caller cannot forget to say who is asking.

Tests

- The existing suite must pass unmodified — this step changes no behaviour, and a test that needed editing would mean it did.
- New: a reflection test that enumerates every public method on the service classes and asserts each one is guard-wrapped. This is the test that stops step 5 from quietly reintroducing a bypass.
- New: guard writes an audit row on success and on failure; consumes a dual-control approval before the handler runs, not after; records metrics on both paths.

Gate

All 816 existing tests pass with no test file modified, and the reflection test proves no service method escapes the guard.

Closes

Nothing directly — this step exists so that the steps after it can be built safely.

Step 2 — Query and reporting layer

Stage A · ~400 lines, ~20 tests.

Let the store answer the questions a console asks, in one query rather than N+1.

Impact on the existing design

- metadata/store.py gains paged query methods; the existing locate() is untouched, so the KMIP handler is unaffected.
- New indexes on the sortable columns — the first schema migration since Phase 1.

Design

- store.locate_page(filters, sort, cursor, limit) returning full rows, an opaque cursor and a total — not the bare identifiers Locate returns.
- Cursor is (sort key, uid) encoded, not an offset: offset paging over a table that is being written double-shows rows.
- store.inventory_report() and store.algorithm_usage() for the two reporting gaps.

Implementation

- Add the new methods beside the existing locate(), which stays exactly as it is for the KMIP handler.
- Indexes to match the sort columns the console offers.

Tests

- Paging over a table mutated between pages neither duplicates nor skips a row — insert and delete between fetches and assert the union.
- Sort by each supported column, ascending and descending; total equals a brute-force count; a tampered cursor is rejected rather than trusted.
- Inventory and algorithm counts match a brute-force scan of the same store.

Gate

A store holding 12,000 objects returns page one of fifty, sorted by expiry, in a single query, with the same figures a full scan produces.

Closes

- Key inventory and discovery
- Crypto-agility reporting

Step 3 — HTTP transport, sessions and service accounts

Stage B · ~900 lines, ~35 tests.

An authenticated HTTPS listener that shares nothing with the KMIP port, and cannot be flooded.

Impact on the existing design

- A second listener and a second authenticated entry point: the security surface roughly doubles.
- New config section, new tables for tokens, new failure modes the KMIP path never had (429, CSRF).
- observability.py keeps its own unauthenticated port, deliberately separate.

Design

- New api: configuration section — own port, TLS mandatory, off by default. Not the observability port: /metrics and /ready are deliberately unauthenticated and must stay that way.
- Opaque 256-bit tokens stored as SHA-256 hashes, with identity, label, created / expires / last used / revoked. Not JWTs: revocation matters more than statelessness, and there is already a database.
- Browsers get an httpOnly, Secure, SameSite cookie plus a CSRF token; machine clients send a bearer header. Long-lived labelled tokens are the service-account mechanism.
- A bounded connection semaphore and a per-identity token bucket.

Implementation

- kmip_pkcs11/api/ with a small stdlib router on ThreadingHTTPServer — the project has three runtime dependencies and none of them does cryptography, which is a property worth keeping.
- Middleware order: authenticate, rate limit, route, translate exceptions into RFC 9457 problem+json.
- POST and DELETE /api/v1/session, GET /api/v1/me.

Tests

- Login succeeds and fails correctly; a disabled identity is refused; a deleted identity's live session stops working immediately.
- Token revocation takes effect on the next request; absolute and idle expiry; a cookie request without a CSRF token is refused while a bearer request is not.
- Rate limiting returns 429 and recovers; the connection cap holds under 200 concurrent connections; the KMIP port is unaffected throughout.
- Configuration validation refuses api.enabled without TLS.
- Every login, logout, token issue and token revoke is an audit row.

Gate

Two hundred concurrent connections do not exhaust the listener, a revoked token is refused on its next use, and the KMIP port serves normally while the API is under load.

Closes

- API keys / service accounts
- Connection and rate limiting
- Request bounds and DoS resistance

Step 4 — Read endpoints

Stage B · ~700 lines, ~40 tests.

Everything a console displays, with authorization identical to KMIP.

Impact on the existing design

- Read paths gain an HTTP representation; the KMIP path does not change at all.
- The 404-versus-403 decision becomes an API contract every later step must preserve.

Design

- GET /keys (paged, filtered, sorted), /keys/{uid}, /keys/{uid}/attributes, /audit, /audit/verify, /approvals, /reports/expiring, /reports/inventory.
- 404 and 403 are chosen deliberately: an object the caller may not see returns the same answer as one that does not exist, so the API does not become an existence oracle.

Implementation

- Read-only handlers calling the services from step 1 — no store access from the transport layer, which the guard reflection test enforces.

Tests

- Authorization parity: for a given identity, GET /keys returns exactly the set KMIP Locate returns. Parametrised over owner, admin, grantee, group member and stranger.
- The paging contract from step 2 holds through the HTTP layer.
- Audit filters by identity, object, result and time range; the approval queue lists pending requests only.
- A read of an object the caller may not see is indistinguishable from a read of one that does not exist.
- Every read is audited — 'who exported this key' is a read.

Gate

A read-only console renders key inventory, audit search and the approval queue against real data, and a test proves no write path is reachable through any registered route.

Closes

Nothing directly — this step exists so that the steps after it can be built safely.

Step 5 — Write endpoints, and the governance gaps

Stage B · ~1,350 lines, ~68 tests.

Lifecycle and delegation over REST, with governance applying by construction rather than by remembering.

Impact on the existing design

- Write paths over HTTP, with governance applied through the guard rather than re-implemented beside it.
- lifecycle/governance.py extends to key pairs, create_split_key gains a second method, and the service layer gains transactions.
- The server acquires a data plane. Until now every caller fetched a key reference and asked the token to act on it; now applications send data and never see a key at all.

Design

- POST /keys; POST /keys/{uid}/activate | revoke | rekey; DELETE /keys/{uid}; PUT /keys/{uid}/cryptoperiod; grants, groups and role permissions; POST /approvals/{id}/approve.
- A dual-control refusal returns 403 carrying the approval request id and the count outstanding, so the console can render 'awaiting 2 approvals' and link to the queue. That falls out of the existing enforcement.
- Three matrix gaps are closed here because they are lifecycle work: asymmetric auto-rotation, Shamir threshold split keys, and batch atomicity via a service-level transaction.
- The data plane beside the control plane: POST /keys/{uid}/encrypt | decrypt | sign | mac, and /datakey returning a fresh key both wrapped under the named key and in the clear — the surface applications actually consume, and the primitives already run inside the token.

Implementation

- All writes go through the guard. Scheduled rotation extends to key pairs using the existing ReKeyKeyPair handler; CreateSplitKey gains a Shamir method alongside XOR; the service layer wraps a batch in one transaction so a mid-batch failure rolls back.
- The crypto endpoints are thin: they translate JSON to the arguments the existing Encrypt, Decrypt, Sign and MAC handlers already take. Re-wrap is decrypt-then-encrypt under the successor key, inside one guarded call so plaintext never leaves the process.

Tests

- The dual-control assertion from the KMIP wire test, run verbatim against REST: the first Destroy is refused, the key survives, two other identities approve, the retry succeeds, and the approval is spent.
- Role allowlists and group grants behave identically over both transports — parametrised over the transport, one test body.
- Asymmetric rotation cross-links both new objects to both old ones.

- Shamir: any k of n shares reconstruct, any $k-1$ fail.
- A batch whose third item fails leaves the first two unapplied.
- Ciphertext from the REST encrypt endpoint decrypts over KMIP and the reverse — one test body, both transports, so the two cannot diverge.
- A data key returns a wrapped form that unwraps to the plaintext form; re-wrap to the successor key yields the original plaintext and the old ciphertext no longer decrypts under the new version.

Gate

The KMIP dual-control test and the REST dual-control test share one body and both pass.

Closes

- Automatic rotation
- Split knowledge / M-of-N shares
- Bulk and batch operations
- Encryption as a service (data-plane API)

Step 6 — Administration, OpenAPI, separation of duties

Stage B · ~900 lines, ~50 tests.

Complete the API surface and split the administrator so no one role can both grant access and use it.

Impact on the existing design

- The admin role splits in two — the first change to the authorization model since Phase 5, and the first needing a migration for existing deployments.
- An OpenAPI document becomes a build artefact CI has to keep in step with the router.

Design

- Identity, role and group administration endpoints.
- The admin role splits into security-admin (identities, roles, policy) and key-admin (objects, cryptoperiods), so a key administrator cannot grant themselves access to what they administer.
- An OpenAPI 3.1 document generated from the router's route table, not hand-written — a hand-written spec drifts within a month.
- An interactive explorer served from that document, and client libraries generated from it in CI rather than written by hand, for the same reason: anything maintained separately from the router drifts from it.

Implementation

- Migration assigns both new roles to anyone holding admin today, so an upgrade changes nobody's access until an operator splits them.

Tests

- A key-admin cannot create an identity; a security-admin cannot read key material; neither can escalate to the other.
- The generated OpenAPI document validates against the 3.1 schema, and a reflection test asserts every registered route appears in it.
- A client generated from the document round-trips a create, read and delete against a live server, so the specification is proved usable and not merely well-formed.
- Responses for a sample of endpoints match their declared schemas.

Gate

openapi.json validates, covers every route the router knows about, and the two admin roles are provably disjoint in capability.

Closes

- REST / JSON API
- Separation of duties
- OpenAPI specification and generated clients

Step 7 — Web console and self-service

Stage B · ~2,000 lines of front-end, ~30 tests.

The interface most of the market considers table stakes, and the reason the API exists.

Impact on the existing design

- A front-end build enters the repository — the first artefact that is neither Python nor documentation tooling.
- Content Security Policy and cookie handling become part of the security review surface.

Design

- A single-page client of the API. No privileged path: the console can do nothing the API forbids, and holds no credential the API would not accept from curl.
- Views: key inventory, key detail, approval queue, audit search, expiry report, identity and role administration.
- Self-service: a team member creates and rotates keys within their group's grants, without an administrator.

Implementation

- Served by the API listener or a CDN. Strict CSP with no unsafe-inline. Session in an httpOnly cookie, never in localStorage.

Tests

- End-to-end with a headless browser (Playwright is available): log in, list keys, open the approval queue, approve a request, watch a blocked Destroy succeed on retry.
- The CSP contains no unsafe-inline and no unsafe-eval.
- A route the API forbids to this identity is not reachable from the console by any means, including direct navigation.

Gate

An operator completes a full day of routine work — provisioning, approving, investigating an audit question — without touching the CLI.

Closes

- Web console
- Self-service developer portal

Step 8 — Enterprise identity and policy

Stage C · ~900 lines, ~40 tests.

Authenticate against the corporate directory, and express conditions the role model cannot.

Impact on the existing design

- Identity resolution gains a third path beside password and mTLS certificate.
- The guard gains a policy evaluation point — the first change to its order of checks since step 1.

Design

- OIDC: validate the ID token (issuer, audience, expiry, signature) and map a claim to a provisioned identity. LDAP/AD bind as an alternative.
- The mTLS path already proves the pattern — an external assertion resolved to a provisioned principal, never inventing one.
- Attribute-based conditions evaluated in the guard before the allowlist: time window, source network, purpose. The engine may only narrow.

Implementation

- Directory group to role mapping, refreshed on login.
- Policies stored in the database, versioned, and every evaluation that denies is audited with the rule that fired.

Tests

- An expired, wrong-audience, wrong-issuer or unsigned token is refused; a valid token for an unprovisioned subject is refused.
- Directory group changes take effect on next login.
- A policy denies outside its window and permits inside it.
- Property test: for any identity and any policy set, the permitted operation set is a subset of what the role model alone would allow. A policy that widens access is a bug by construction.

Gate

A corporate login yields exactly the roles the directory says, and a policy denial names the rule that fired in the audit log.

Closes

- Enterprise IdP — LDAP/AD, SAML, OIDC
- Attribute or policy-based access

Step 9 — Audit externalisation, compliance and ceremony

Stage C · ~1,350 lines, ~60 tests.

Make the audit trail survive an attacker with file access, and produce the evidence an auditor asks for.

Impact on the existing design

- The audit log stops being purely local: it gains an outbound path and an external dependency.
- Worker 0 takes on a third single-instance responsibility beside the scheduler and the metrics endpoint.
- Startup gains a sealed state. Every health check, every deployment script and every operator runbook has to account for a service that is running but not yet open.

Design

- A shipper emitting audit rows as syslog/CEF, with a persisted cursor so a restart neither duplicates nor loses entries.
- Periodic notarisation of the chain digest to an append-only external endpoint. This is the one thing the local chain cannot do: detect a wholesale, internally consistent rewrite.
- Compliance report generators over the audit log and key inventory — PCI DSS key-management evidence, NIST SP 800-57 cryptoperiod evidence.
- A key ceremony runbook, and a kmip-admin command producing a signed transcript of what was done, by whom and witnessed by whom — extended to the token PIN itself, split into operator-held shares so the service starts sealed and a quorum opens it. Today one PIN holder is the whole ceremony.
- OpenTelemetry spans around every guarded operation and token call. Metrics say a request was slow; a span says which PKCS#11 call it waited on.
- Alerting rules over the metrics already exported.

Implementation

- The shipper runs in worker 0 alongside the scheduler, with the same single-instance reasoning.

Tests

- The shipper emits every row exactly once across a restart — kill it mid-stream and assert the union at the collector.
- CEF output parses with a standard parser.
- Anchoring detects tampering the local chain cannot: rewrite the whole log consistently, re-chain it so `verify_audit_chain` passes, and assert the anchor comparison fails.
- Report figures match direct store queries.
- A ceremony transcript verifies against its signature and fails after a single byte is changed.

- Any k of n unseal shares start the service and any $k-1$ leave it sealed; a sealed service answers /health and refuses every KMIP and REST operation rather than failing obscurely deep in the shim.
- A traced request carries one trace id from the HTTP layer through the guard to the token call, and a sampled-out request costs no spans.

Gate

A rewritten but internally consistent audit log — one that passes local verification — is caught by the external anchor.

Closes

- Formal key ceremony
- External audit anchoring
- SIEM integration
- Compliance reporting
- Alerting
- Distributed tracing
- Sealed startup with quorum unseal

Step 10 — Storage abstraction and PostgreSQL

Stage D · ~1,500 lines, ~60 tests. Needs a PostgreSQL instance in CI.

Remove the single-writer constraint that blocks everything in stage D.

Impact on the existing design

- metadata/store.py stops being SQLite and becomes an interface with two implementations — the largest structural change in the plan.
- Every SQLite-specific mechanism needs a deliberate equivalent, including the lock that stops the audit chain forking.

Design

- Extract a Store interface; SQLite becomes one implementation and PostgreSQL another.
- Each SQLite-specific mechanism needs a deliberate equivalent, not a translation: PRAGMA user_version becomes a schema-version table, json_extract becomes JSONB operators, RAISE(ABORT) triggers become PL/pgSQL triggers, and BEGIN IMMEDIATE — which is what keeps the audit chain from forking — becomes an advisory lock or SERIALIZABLE.

Implementation

- Both backends ship. SQLite stays the default for single-node deployments; it is a good answer there and removing it would be a regression.

Tests

- The entire existing store suite runs against both backends from one parametrised fixture — not a parallel copy that drifts.
- The audit chain survives concurrent writers on PostgreSQL: hammer it from several connections and assert the chain verifies and no sequence number is reused.
- Migrations apply identically on both; backup and restore work on both.
- A store dump from one backend restores into the other.

Gate

The full suite — 816 tests plus everything added since — passes against PostgreSQL as well as SQLite.

Closes

- Enterprise database backend

Step 11 — High availability, failover and deployment

Stage D · ~1,800 lines, ~55 tests. Needs multiple nodes and a second token.

Survive the loss of a node, a token, or a site.

Impact on the existing design

- Single-process assumptions become cluster-wide ones: the scheduler needs a leader lock, not a worker index.
- pkcs11_shim gains a pool and health checks — the first change to the shim's contract since the capability probe.

Design

- Several nodes against one PostgreSQL. The scheduler takes a leader lock so exactly one instance enforces cryptoperiods — the same reasoning that restricts it to worker 0 today, one level up.
- An HSM pool: several tokens, health-checked, with failover. The master key must exist on each, which makes provisioning part of the design rather than an afterthought.
- Scheduled backup with offsite shipping; Helm chart and manifests; the container image and CI workflow finally executed.

Implementation

- Read traffic spreads across nodes; audited writes still serialise on the chain, so this raises availability more than throughput. Say so rather than implying linear scaling.

Tests

- Kill the leader under load: another takes over within the lease period, and no key is deactivated twice.
- Fail a token mid-operation: the request either completes on another or fails cleanly, never silently half-applies.
- Two nodes serving concurrently produce one intact audit chain.
- A scheduled backup taken on one node restores on another.
- The container image builds and its health check passes — the first time this has been executed anywhere.

Gate

A node is killed under sustained load: no request is lost, the audit chain verifies, and cryptoperiod enforcement continues uninterrupted.

Closes

- HA clustering
- Multi-site replication and DR
- HSM failover and pooling
- Horizontal throughput
- Scheduled and offsite backup
- Kubernetes / container deployment

Step 12 — Multi-tenancy

Stage D · ~1,600 lines, ~80 tests.

Let unrelated tenants share one deployment without seeing each other.

Impact on the existing design

- A tenant column reaches every table, every query, the audit log, the CLI and the API.
- Object names stop being globally unique — the change most likely to surprise an existing deployment.

Design

- Tenant becomes a first-class column on objects, identities and audit rows, and every query is scoped by it.
- Object names become unique per tenant rather than globally — the change most likely to surprise an existing deployment.
- Quotas: object count, operations per second, storage. Per-tenant audit views. Tenant administrators who cannot see other tenants.
- This is why it is last: the boundary reaches into every query, the audit log, the CLI and the API, and half of it would be worse than none.

Implementation

- A migration places every existing object in a default tenant, so a single-tenant deployment behaves exactly as before.

Tests

- Exhaustive isolation: parametrised over all 41 KMIP operations and every REST route, an identity in tenant B cannot read, modify, locate or destroy an object in tenant A. This is the test that matters and it should be generated from the operation table, not hand-written.
- Name collisions across tenants are allowed; within a tenant they are not.
- Quota enforcement at the boundary, and the audit row that records it.
- A tenant administrator cannot escalate out of their tenant.
- Audit search returns only the caller's tenant.

Gate

A generated test over every operation and every route proves no cross-tenant reachability exists.

Closes

- Multi-tenancy and namespaces
- Per-tenant quotas

Step 13 — KMIP JSON and XML encodings

Stage E · ~700 lines, ~40 tests — mostly the existing corpus re-parametrised.

Let a client speak KMIP without implementing a TTLV codec.

Impact on the existing design

- core/ttlv.py stops being the only codec, and the tag registry becomes shared infrastructure rather than a TTLV detail.
- Nothing above the codec changes. That is the point of the step: the service layer cannot tell which encoding a request arrived in.

Design

- The object model does not change. What changes is the wire: a JSON codec and an XML one beside core/ttlv.py, both driven by the same tag and type tables, so a new tag is still added in one place.
- An HTTPS binding on the API listener — POST /kmip, encoding chosen by Content-Type — rather than a second socket. The KMIP TCP port is untouched.
- Encoding is a transport concern, so it sits below the service layer: the same guard, the same handlers, the same audit rows.

Implementation

- core/encodings/ with json.py and xml.py implementing the OASIS profiles, sharing the tag registry the TTLV codec already reads.
- Content negotiation, and 415 for an encoding that is not enabled. Both are off by default: a new parser is new attack surface.

Tests

- Round-trip every operation's request and response through all three encodings and assert the decoded structures are identical — the existing conformance corpus re-run twice, not a new set of assertions.
- The OASIS JSON and XML test vectors decode to what the TTLV vectors decode to.
- A batch, a wrapped key and a dual-control refusal behave the same over JSON as over TTLV, so an encoding cannot change semantics.
- An unknown tag, a truncated document and a type mismatch are refused with the same error the TTLV codec raises.

Gate

The conformance suite passes unchanged when re-run against the JSON and XML encodings, and a request refused over TTLV is refused identically over both.

Closes

- KMIP JSON and XML encodings

Step 14 — Certificate discovery and expiry monitoring

Stage E · ~600 lines, ~30 tests.

Know which certificates the estate is actually running, and warn before one expires.

Impact on the existing design

- The server starts reaching outward. Everything else here waits to be called; this initiates connections, which is a new posture for the service and for its firewall rules.
- The inventory stops being a view of what this KMS holds and becomes a view of the estate, including certificates it does not manage.

Design

- A scanner that connects to configured TLS endpoints, records the chain presented, and stores subject, issuer, fingerprint and validity window against the inventory.
- Certificates registered here are matched by fingerprint, so the report separates what this KMS manages from what it does not — the second number is the one that matters.
- Expiry thresholds feed the alerting rules built in step 9 rather than opening a second notification path.

Implementation

- `kmip_pkcs11/discovery/` with the scanner, a scheduler entry beside the cryptoperiod scheduler in worker 0, and the inventory tables.
- Scanning is read-only and rate-limited. This is a tool that touches production endpoints, and it should be incapable of harming them.

Tests

- A scan of local TLS servers with known certificates records exactly those chains, intermediate included.
- A certificate registered here and the same certificate found by scanning reconcile to one inventory row, matched on fingerprint.
- An endpoint that times out, presents an expired certificate, or drops the handshake is recorded as such rather than skipped silently.
- An expiry threshold fires once per certificate per window, not on every scan.

Gate

A scan of a fixture estate reports every certificate, splits managed from unmanaged correctly, and raises an expiry alert exactly once for a certificate inside the warning window.

Closes

- Certificate discovery and expiry monitoring

Step 15 — Cloud BYOK

Stage F · ~900 lines, ~40 tests. Provider SDKs are the first runtime dependencies this project would add – worth resisting; each import API is a signed HTTPS call.

Generate here, import into a cloud KMS, keep custody of the source.

Impact on the existing design

- The server gains outbound calls to third-party services, with credentials to manage and failures to handle.
- A key can now exist in two places at once. The object model has to say which is authoritative, and it says this one.

Design

- A connector per provider calling its import API: AWS ImportKeyMaterial, Azure Key Vault key import, Google Cloud KMS ImportJob. Each wants the key wrapped to a public key it issues, which the existing KeyWrappingSpecification already produces.
- The source object stays here and gains a link to what it became, so the inventory can answer where a key ended up — the question an auditor asks about BYOK and the one most implementations cannot.
- Configuration, not code, names the clouds: credentials come from the environment, never the config file.

Implementation

- `kmip_pkcs11/cloud/` with one module per provider behind a shared interface, driven from the service layer so the guard, audit and role checks apply to an export-to-cloud exactly as to any other disclosure.

Tests

- Each provider's wrapping format is produced correctly, checked against its published test vectors rather than against our own encoder.
- An import failure leaves no partial state and no link claiming success.
- Export-to-cloud is refused for an identity without the right, and audited on both the success and the refusal.
- The link survives rotation: rotating the source key does not orphan the record of what was imported where.

Gate

A key generated in the token is imported into all three clouds, and the inventory names where each copy lives.

Closes

- Cloud BYOK (AWS, Azure, GCP)

Step 16 — External key store — XKS and EKM

Stage F · ~1,100 lines, ~50 tests.

Let a cloud service call back here for every operation, so the key never leaves at all.

Impact on the existing design

- The server becomes latency-critical for someone else's workload — a different operational posture from a key manager clients poll.
- Availability stops being our own concern only: an outage here is an outage in the cloud service that depends on it.

Design

- The AWS XKS proxy API and the Google Cloud EKM API as HTTPS services in front of the same service layer. Both are small, well-specified request/response contracts over operations this server already performs.
- This is the strongest custody story the product can offer: the cloud holds ciphertext and calls out to decrypt, so revoking access here revokes it everywhere, immediately.
- Latency and availability become the cloud's problem and therefore ours: an XKS endpoint that is slow fails the caller's operation.

Implementation

- `kmip_pkcs11/api/xks.py` and `ekm.py` sharing the API listener, the authentication and the rate limiter built in step 3.
- Request signing verified per the provider's scheme before anything reaches the service layer.

Tests

- The published XKS conformance cases pass, including the error codes — a wrong code fails a caller in a way that looks like data loss.
- A request with a bad signature is refused before the token is touched.
- Revoking a key here makes the cloud's next operation fail, and the audit log shows why.
- Latency under sustained load stays inside the provider's timeout, measured rather than assumed.

Gate

A cloud service encrypts and decrypts through this server for a full test run, and revoking the key stops it mid-run.

Closes

- Cloud EKM / XKS / hold-your-own-key

Step 17 — PKCS#11 provider for applications

Stage F · ~2,000 lines in a new language, ~50 tests. The largest single step here and the one most worth deferring until a caller asks for it.

Let applications that speak PKCS#11 use this server without knowing KMIP exists.

Impact on the existing design

- A second deliverable, in another language, with its own release cycle and its own memory-safety burden.
- The KMIP surface becomes an internal contract for it, so changes there can break a client nobody is testing in this repository.

Design

- A shared library exposing the PKCS#11 C interface and translating to KMIP over the wire — the inverse of `pkcs11_shim/`, and the reason that boundary was kept clean.
- A deliberate subset: `C_GenerateKey`, `C_Encrypt`, `C_Decrypt`, `C_Sign`, `C_Verify`, `C_FindObjects` and session handling. Implementing the whole of PKCS#11 badly would be worse than implementing part of it well and saying which part.
- Session and object handles map to KMIP identifiers in the library, so the server stays stateless per request.

Implementation

- A separate deliverable in its own directory with its own build — C, or Rust with a C ABI. It is a client of this server, not part of it.
- The subset is declared in its documentation and the library returns `CKR_FUNCTION_NOT_SUPPORTED` honestly for the rest.

Tests

- `pkcs11-tool` exercises every implemented function against a live server.
- A Java keystore and an OpenSSL engine both load the library and complete a sign and a verify.
- Every unimplemented function returns `CKR_FUNCTION_NOT_SUPPORTED` rather than crashing or silently doing nothing.
- Handle exhaustion and concurrent sessions behave — this is C, so the tests include a leak check under repeated open and close.

Gate

An unmodified application that speaks PKCS#11 signs with a key it cannot extract, and `pkcs11-tool` reports the supported mechanism list.

Closes

- PKCS#11 provider for applications

Step 18 — Tokenization and format-preserving encryption

Stage F · ~700 lines, ~40 tests.

Protect PAN and PII in place, where the ciphertext has to look like the plaintext.

Impact on the existing design

- The first cryptographic primitive implemented in this codebase rather than delegated to the token — which is exactly the property the project has protected so far, and the reason to keep it isolated behind its own module.
- A vault of reversible mappings is a new kind of asset to protect, with different backup and residency implications from a key.

Design

- FF1 from NIST SP 800-38G as a service in front of the token: the tokenising key stays in the HSM and the format transformation happens here.
- A token vault for the detokenisation mapping where format preservation is not enough, with the same lifecycle and audit as any other object.
- SP 800-38G's own guidance about small domains is enforced rather than documented: a domain below the safe size is refused, not warned about.

Implementation

- `kmip_pkcs11/fpe/` as a service-layer module reachable over REST. Not a KMIP operation — the protocol has no vocabulary for it, and inventing vendor extensions would interoperate with nothing.

Tests

- FF1 matches the NIST test vectors exactly. This is the whole correctness argument for the step and nothing else substitutes.
- Round-trip over every supported alphabet and length, including the boundaries.
- A domain too small to be safe is refused with an error naming the limit.
- Detokenisation is access-controlled and audited as a disclosure, because that is what it is.

Gate

The NIST FF1 vectors pass, and a PAN tokenises to something that is still a valid PAN and detokenises only for an identity allowed to.

Closes

- Tokenization / format-preserving encryption

Step 19 — Vendor interoperability validation

Stage G · ~400 lines of fixes and harness, ~60 recorded-exchange tests. Mostly not code: it is scheduling, licences and patience.

Turn two rows that say “should work” into two rows that say “does, against these products, at these versions”.

Impact on the existing design

- The conformance suite stops being only our reading of the specification and starts carrying other implementations' behaviour.
- Supported-version claims become a maintenance commitment — a validated product is one someone expects to keep working.

Design

- A validation matrix: product, version, profile, what was exercised, what had to change. Published with the results, including the failures.
- Each vendor profile makes assumptions the specification does not state. The deliverable is the list of them, and a conformance test per assumption so it cannot regress.
- Order by what is asked for most: Oracle TDE and VMware first.

Implementation

- No new subsystem — fixes land in the operations the profiles exercise, and every exchange captured becomes a test in the conformance suite.
- A recorded-exchange harness so a vendor's traffic can be replayed without the vendor present.

Tests

- A captured exchange from each validated product replays green in CI forever after, which is what stops this work decaying.
- Profile-specific attribute handling is asserted per vendor rather than in general.
- The matrix itself is generated from the test results, so it cannot claim a product that is not tested.

Gate

At least one database and one storage product complete their full key lifecycle against this server, with the exchanges replaying in CI.

Closes

- Database TDE integration
- Storage, backup and VM integrations

Step 20 — OASIS interoperability certification

Stage G · ~200 lines of fixes, plus the event. Small in code, long in calendar.

Demonstrate interoperability against other implementations, in public, rather than asserting it.

Impact on the existing design

- External parties see the implementation's behaviour directly. Anything the specification leaves ambiguous and we resolved by assumption surfaces here.
- A published result is a commitment: it dates, and it has to be redone as the protocol moves.

Design

- Run the OASIS KMIP conformance test cases end to end and publish the results, including anything that fails.
- Participate in an interop event against other vendors' clients and servers — the part that cannot be simulated, because the value is precisely that the other implementation is not ours.
- The conformance cases are already mapped in the suite; what is missing is the execution and the counterparty.

Implementation

- Fixes arising from the event, and the test cases promoted into the standing suite.

Tests

- Every mapped OASIS conformance case executes as part of CI rather than being mapped and unrun.
- Findings from the event become regression tests before the fix lands.
- The published result names the profile and version tested, so a reader can tell what the certification covers.

Gate

The conformance suite runs clean, and an interop event produces a result that can be published as it stands.

Closes

- KMIP interoperability certification

Step 21 — Certificate authority integration

Stage G · ~600 lines, ~35 tests.

Get real certificates without this server pretending to be a certificate authority.

Impact on the existing design

- An external dependency on the issuance path — the first place where an outage elsewhere blocks an operation here.
- The row closes by integration rather than by implementation, which is the honest way to close it and should be read that way.

Design

- Front the deployment with an established CA — EJBCA is the reference — and make Certify and ReCertify proxy to it: the key stays in the token, the CSR goes out, the issued certificate comes back as a managed object.
- CRL and OCSP become the CA's job, and this server records and serves revocation status rather than deciding it.
- The alternative — implementing a CA here — was rejected earlier in this project and is rejected again. A weak CA inside a key manager invites exactly the reliance it cannot support.

Implementation

- A CA backend interface with an EJBCA implementation, reached from the Certify and ReCertify handlers so both transports get it.
- Self-signed issuance stays for development, behind a configuration flag that is off by default and logs loudly when on.

Tests

- An issued certificate chains to the CA and validates against it with a standard tool, not with our own validator.
- Revoking through the CA is reflected here, and Validate refuses the revoked certificate.
- A CA outage degrades honestly: issuance fails with a clear error, existing objects keep working.
- The development self-signed path cannot be enabled without it appearing in the audit log.

Gate

A certificate issued through this server chains to a real CA, validates with openssl verify, and stops validating once revoked.

Closes

- Certificate lifecycle

Step 22 — Secrets manager integration

Stage G · ~500 lines, ~30 tests.

Serve dynamic secrets without becoming a secrets manager.

Impact on the existing design

- The scheduler gains a second class of expiry to enforce, beside cryptoperiods.
- Like step 21, this closes by integration. The product boundary does not move; the deployment story does.

Design

- The same shape as the CA step: an established secrets manager handles dynamic issuance, templating and leases, and uses this server for the keys it protects them with.
- What this server adds is the half it should own — short-lived SecretData with automatic destruction at lease end, and the audit trail joining a lease to the identity that held it.
- Rejected again, for the third time in this plan: building an engine that mints database credentials. That needs deep knowledge of each backend and is a different product.

Implementation

- Lease expiry wired to automatic destruction in the existing scheduler, and a documented integration pattern with a reference configuration.

Tests

- A lease that expires destroys its secret, and the destruction is audited against the original requester.
- An expired lease cannot be renewed into existence.
- The reference integration completes a full issue, use and expire cycle against a real secrets manager.
- Clock changes do not resurrect an expired lease or destroy a live one.

Gate

A secret issued under a lease is unreadable and gone after expiry, with an audit trail a reader can follow from request to destruction.

Closes

- Secrets management

Step 23 — KMIP 3.0

Stage H · ~1,200 lines, ~70 tests.

Move to the current specification line, which is also where post-quantum lives.

Impact on the existing design

- The protocol layer becomes version-aware, which touches the dispatcher and every enumeration table.
- Encapsulate and Decapsulate exist and work on classical algorithms, so the only thing missing for post-quantum is the token.

Design

- The 3.0 additions on top of 2.1: new enumerations, the Encapsulate and Decapsulate operations, and the protocol version negotiation that lets a 2.1 client keep working unchanged.
- Version negotiation is the part that carries risk. An existing deployment must not notice this step happened.
- Entirely in-protocol: no new dependency, no new transport, no schema change.

Implementation

- `core/enums.py` extends, new operation handlers join operations/, and the dispatcher selects behaviour by negotiated version rather than assuming 2.1.

Tests

- Every existing 2.1 test passes unchanged against a 3.0-capable server — this is the gate, and a modified test would mean the step broke compatibility.
- A 2.1 client and a 3.0 client are served correctly by one server at the same time.
- Encapsulate and Decapsulate round-trip, tested with a classical algorithm so the operations are proved before a PQC token exists.
- Version downgrade attempts are rejected rather than silently honoured.

Gate

The full 2.1 suite passes against the 3.0 server with no test modified, and both protocol versions are served concurrently.

Closes

- KMIP 3.0

Step 24 — Post-quantum and full algorithm breadth

Stage H · ~800 lines, ~45 tests. Small, because the probe architecture did the work years earlier.

Run ML-KEM, ML-DSA and SLH-DSA, and light up the algorithms the current token cannot perform.

Impact on the existing design

- Little changes in the code. The change is in what the deployment can claim, which is the point of the row.
- Hybrid objects are the one genuinely new modelling problem, and the one most likely to be got wrong quietly.

Design

- No new design. The capability probe already gates every algorithm on what the token advertises, so a PQC-capable token activates these the moment it is configured — the work is proving it, not building it.
- Hybrid modes, where a classical and a post-quantum algorithm are used together, need explicit modelling: two keys, one logical object, and a lineage that survives retiring either half.
- Migration guidance matters more than the algorithms here: an estate moves to PQC over years, and the crypto-agility report from step 2 is what makes that planning possible.

Implementation

- Algorithm and mechanism mappings for the FIPS 203, 204 and 205 families, and hybrid key-pair objects in the object model.

Tests

- The full suite runs against the PQC token with the new algorithms exercised end to end, not merely advertised.
- Encapsulate and Decapsulate against ML-KEM interoperate with a reference implementation, not only with ourselves.
- A hybrid object survives rotation of either half with lineage intact.
- The capability probe degrades correctly: on a token without these, requests are refused cleanly and everything else still works.

Gate

The suite passes on a PQC-capable token with ML-KEM and ML-DSA exercised, and unchanged on a token without them.

Closes

- Post-quantum (ML-KEM, ML-DSA, SLH-DSA)
- Algorithm breadth

Step 25 — Validated hardware migration

Stage I · ~600 lines, ~40 tests, plus the hardware and the validation reading.

Move to a token that is certified, and that can enforce policy the server currently enforces for it.

Impact on the existing design

- The strongest single improvement to the product's security posture in this plan, and the one requiring the least code.
- It also tests the project's central architectural bet — that the PKCS#11 boundary makes the token swappable. If that bet is wrong, this is where it shows.

Design

- Run the entire suite against a FIPS 140-3 and Common Criteria validated HSM. The PKCS#11 boundary is what makes this a configuration change rather than a rewrite — and this step is the first time that claim is tested rather than asserted.
- Move dual control into the key where the token supports it. Securosys Smart Key Attributes are the reference: quorum and time-lock carried in the key's own attributes, so compromising this server no longer compromises the policy.
- Server-side dual control stays as the fallback for tokens without it, and the two must not disagree.

Implementation

- A token-capability layer above the shim that uses key-bound policy where offered and the existing enforcement where not, reporting which is in force.

Tests

- The full suite passes on validated hardware with no test modified — any modification means the abstraction leaked.
- With key-bound policy active, an operation is refused by the token even when the server is made to approve it. That is the whole point of the row and the only test that demonstrates it.
- Which enforcement is in force appears in the audit record, so an auditor can tell the two apart.
- Migration moves keys between tokens without exposing material.

Gate

The suite passes on validated hardware, and a deliberately compromised server cannot obtain a key whose policy lives in the token.

Closes

- FIPS 140-2/3 validated HSM
- Common Criteria / eIDAS
- Key-bound usage policy enforced in hardware

Step 26 — Confidential computing deployment

Stage I · ~500 lines, ~25 tests, plus the platform.

Run where the host operator cannot read the process.

Impact on the existing design

- The threat model changes: the host operator becomes an adversary, which is a different assumption from every other step here.
- Attestation makes deployment auditable in a way a configuration file never is.

Design

- Deployment inside a confidential VM or enclave, with remote attestation a client can check before trusting the service.
- The store has to assume an untrusted host: everything sensitive stays encrypted outside the enclave, which the envelope encryption from Phase 1 already mostly provides.
- Attestation is the deliverable, not the VM. A confidential VM nobody verifies is a normal VM with a longer invoice.

Implementation

- An attestation endpoint serving the platform's evidence, and a deployment guide per platform.
- Startup refuses to unseal outside a verified enclave when configured to require one.

Tests

- Attestation evidence verifies against the platform's root, and a tampered measurement fails it.
- The service refuses to start outside an enclave when configured to require one.
- Everything leaving the enclave is encrypted, checked by inspecting what actually lands on the host disk rather than by reading the code — the WAL sidecar in Phase 1 is the precedent for why.

Gate

A client verifies the attestation before connecting, and a modified image fails verification.

Closes

- Confidential computing deployment

5 What the plan waits on

Every one of the 50 open features is routed to a step. That is worth stating carefully, because a plan with a route for everything can read as a plan that can be executed end to end, and this one cannot — not today, and not by adding people.

19 of the 26 steps could start this week. The other 7 are fully specified and blocked on something outside this project: hardware to buy, a product to deploy, or an event someone else runs. They are listed here rather than left for a reader to discover by reaching stage I and finding nothing can begin.

Step	Title	Cannot start until
19	Vendor interoperability validation	Licences for, or access to, the vendor products being validated.
20	OASIS interoperability certification	An OASIS interoperability event, and other implementations to test against.
21	Certificate authority integration	A deployed certificate authority — EJBICA or equivalent.
22	Secrets manager integration	A deployed secrets manager to integrate with.
24	Post-quantum and full algorithm breadth	A PKCS#11 token implementing ML-KEM, ML-DSA and SLH-DSA. SoftHSM's work on these is in the development tree, not in any release.
25	Validated hardware migration	A FIPS 140-3 and Common Criteria validated HSM, ideally one supporting key-bound approval policy.
26	Confidential computing deployment	A confidential-computing platform with remote attestation.

Two of these deserve a second look before they are treated as distant. Step 25 needs a validated HSM, and it is simultaneously the largest security improvement in the plan and among the smallest in code — the cost is procurement, not engineering. Step 24 needs a post-quantum token and nothing else: the capability probe already gates on what a token advertises, so the algorithms activate when the hardware does.

Nothing in this section is a feature the plan declines. The previous revision deferred sixteen; this one routes all of them and is explicit about which cannot yet begin. That is a more useful statement than a deferred list, and a more demanding one.

Six of these — the two cloud integrations, the two vendor validations, secrets management and certificate lifecycle — are ordinary engineering and would form a natural stage E once the REST layer exists to configure them. That would take the total closed to 56 of 50. The remaining eight depend on a supplier, a procurement decision, or an OASIS event, and no amount of planning moves them.

6 Risks worth naming now

Risk	Why it matters	Mitigation
A later endpoint bypasses the guard	All governance silently stops applying to that path, and nothing fails — the worst kind of defect this project has seen.	The reflection test in step 1, kept green forever. It is cheap and it is the single most valuable test in the plan.
Two authenticated front doors	The API doubles the authentication surface, and a leaked token is as good as a password.	Tokens are revocable, expiring, hashed at rest, and every issue and revoke is audited (step 3).
The audit write lock becomes the bottleneck	It already caps worker scaling at about 1.5x; REST traffic contends for the same lock.	Measure before and after step 11. If it binds, per-shard chains are the answer, and that is a design change, not a tuning exercise.
Tenancy migration surprises an existing deployment	Object names become unique per tenant rather than globally.	The step 12 migration places everything in a default tenant so behaviour is unchanged until a second tenant exists.
Scope creep into a security product	Tokenization, a CA and a secrets engine are all adjacent and all tempting.	They are named in section 5 as out of scope. Revisit deliberately, not by drift.
The plan outruns the verification environment	Steps 10 and 11 cannot be honestly completed without PostgreSQL, a second token and a registry.	Section 3 says so up front. A step whose gate cannot be demonstrated is not finished, whatever the code says.

One further note, from this project's own history: every serious defect found so far was invisible in the diff and obvious the moment something ran — an authentication bypass, plaintext in a WAL sidecar, a readiness probe that lied for eighteen seconds, a 25x latency cost, and a slide deck whose every diagram was missing. Each step's gate is written to be executed, not reviewed.